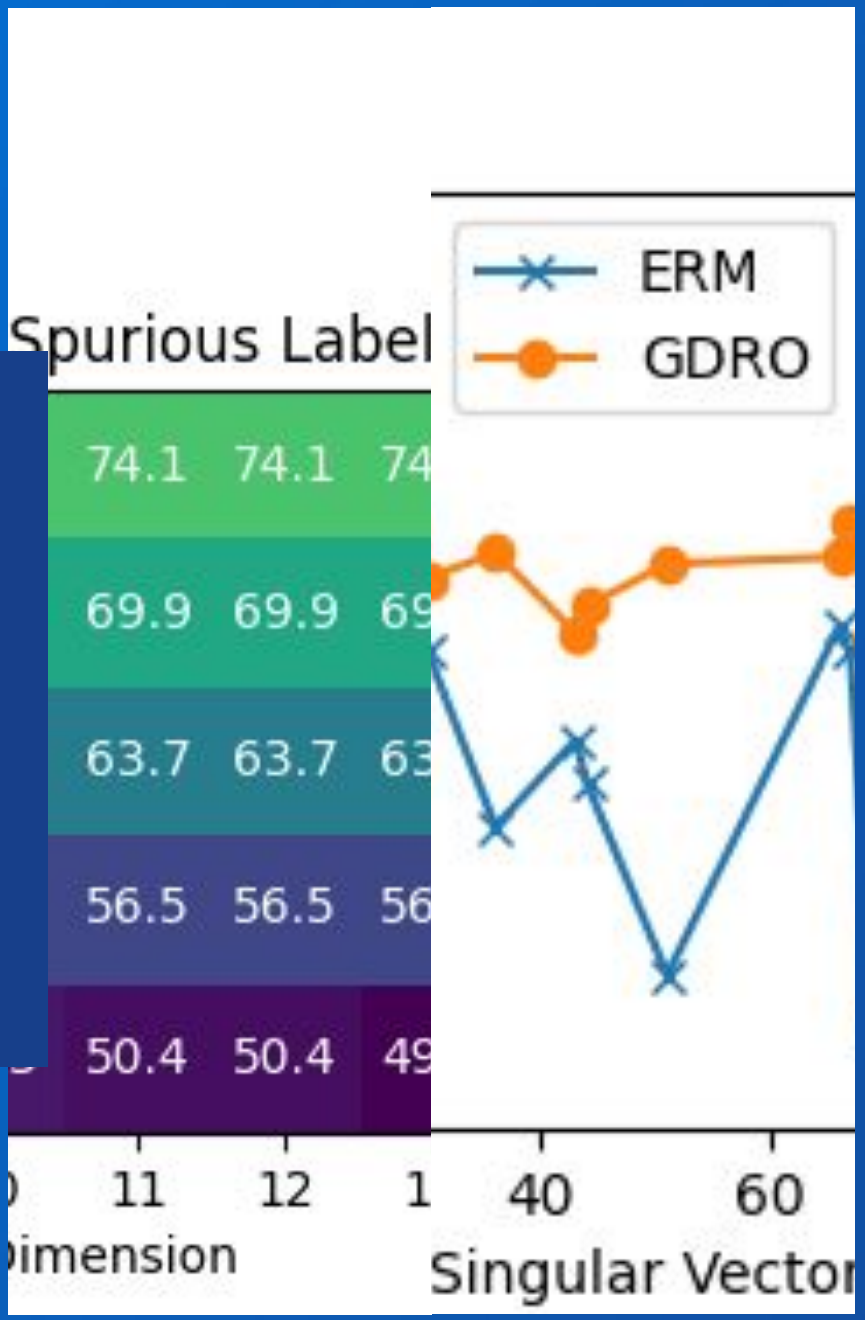
ESCUELA DE INGENIERÍA
FACULTAD DE INGENIERÍA

EDUCACIÓN PROFESIONAL

Guía Resumen Entrenamiento DNNs

Pasos para entrenar una red neuronal en la práctica.

Profesor:
Alain Raymond



4 cosas que hacer en la práctica

1. Determinar Problema a resolver
2. Elegir/Diseñar un modelo adecuado.
3. Entrenar el modelo
4. Evaluar el modelo

1. Determinar Problema a Resolver

- Primero, ¿qué tarea quiero resolver? ¿Cuáles van a ser mis datos de entrenamiento, validación y test?
- Ejemplos:
  - Clasificar fotos de animales por tipo de animal.
  - Identificar ubicación de una persona en una imagen.
  - Registrar en qué momento de un audio se dice la palabra "Hola".
  - Identificar qué acción está ocurriendo en un video en un instante del tiempo
  - Predecir demanda en base a demandas de meses anteriores, precios, etc.
  - Etc…

1. Determinar Problema a Resolver

- ¿Es mi problema de regresión o de clasificación?
  - **Regresión**: tratar de predecir uno o varios valores específicos.
  - **Clasificación**: predecir si entrada corresponde a una de N clases predeterminadas.
- Importante, pues determina la **función de pérdida** que vamos a ocupar para entrenar
- Función de pérdida nos dice **qué tan equivocadas están las predicciones** de nuestro modelo.
  - Regresión → Mean Squared Error (MSE)
  - Clasificación → Cross Entropy

1. Determinar Problema a Resolver

- Datos de Entrenamiento tenemos que ser capaces de:
  - Transformarlos a un formato que entienda el modelo → Tensores.
  - Agruparlos en minibatches para que el modelo haga predicciones.
  - Asegurarnos de ver todos los datos de entrenamiento antes de repetirlos (concepto de época).

2. Elegir/Diseñar un modelo adecuado

- Dado el desarrollo de la tecnología actual, tenemos dos opciones a la hora de decidir qué modelo ocupar.
  - Elegir un modelo ya existente.
    - Usualmente preentrenado
    - Lo alteramos para ajustarlo a nuestras necesidades.
  - Diseñar un modelo desde 0.
    - Costoso, difícil.
    - Usualmente algo que hace gente dedicada a investigación sea en academia o empresas.

2. Elegir/Diseñar un modelo adecuado

- Dado el desarrollo de la tecnología actual, tenemos

¿Pero cómo está estructurada una red neuronal profunda en la práctica?

  - Diseñar un modelo desde 0.
    - Costoso, difícil.
    - Usualmente algo que hace gente dedicada a investigación sea en academia o empresas.

2. ¿Cómo está estructurada una Red Neuronal Profunda?

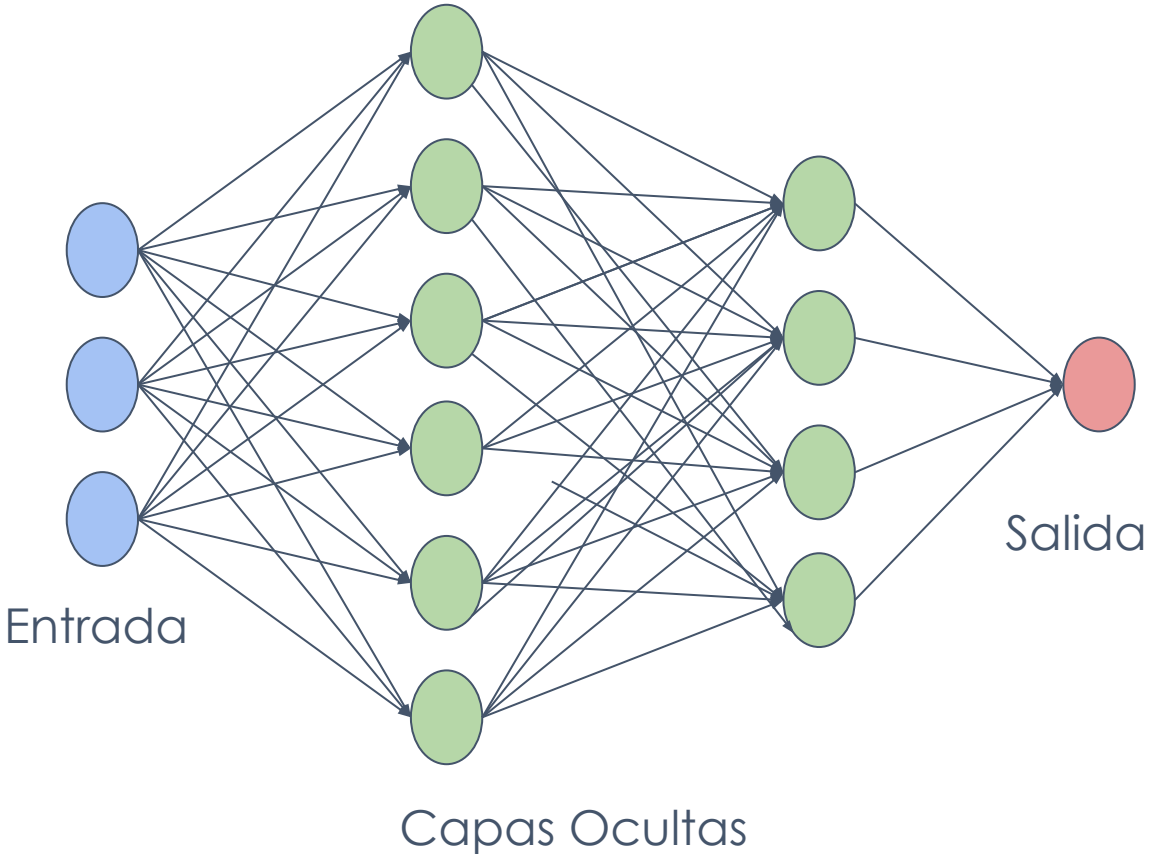
- Una red profunda es una colección de **módulos** que (usualmente) se ejecutan de forma secuencial.
- Cada **módulo** toma como entrada una representación numérica y retorna una **transformación** de esa representación.
  - Esta transformación usualmente altera la dimensionalidad de la representación.

2. ¿Cómo está estructurada una Red Neuronal Profunda?

- ¡Ustedes ya conocen un par de módulos!
  - **Lineal**: ¡módulo usado en los MLP! ¡Todos contra todos!
  - **Sigmoide**: función de activación.
- Los módulos se pueden conectar entre ellos usualmente de manera libre.
  - La única restricción es que calcen las dimensionalidades que espera recibir un módulo con la dimensionalidad de la salida del módulo predecesor.

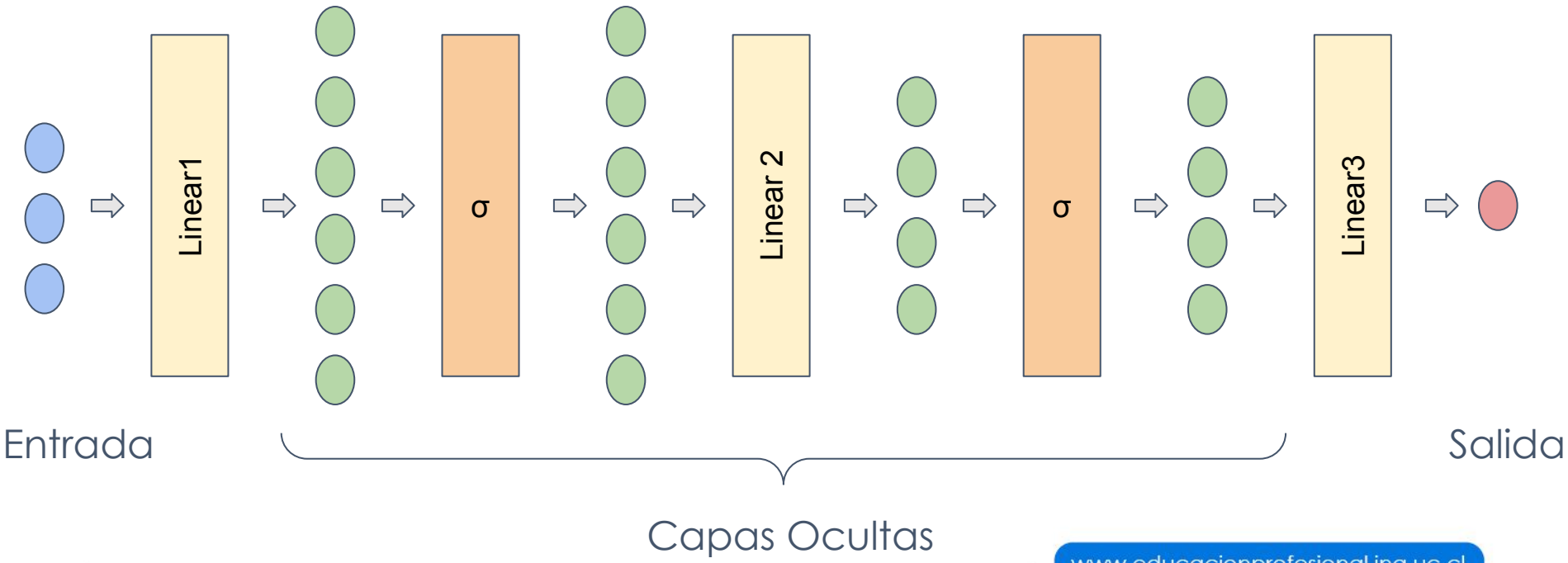
2. ¿Cómo está estructurada una Red Neuronal Profunda?

- **Ejemplo:** Multilayer Perceptron (MLP).
- Entrada de tamaño 3, salida tamaño 1.
- Capas ocultas de 6 y 4 neuronas.
- Función de activación Sigmoide.



2. ¿Cómo está estructurada una Red Neuronal Profunda?

- Un MLP es una secuencia de módulos Lineales con funciones de activación intercaladas.
- En la práctica esto se ve más o menos así:



2. ¿Cómo está estructurada una Red Neuronal Profunda?

- En pseudocódigo se ve así:

```
# x es nuestra entrada y es de tamaño 3
x = Linear1(x)  #x ahora es de tamaño 7
x = Sigmoid(x)  #x ahora es de tamaño 7
x = Linear2(x)  #x ahora es de tamaño 5
x = Sigmoid(x)  #x ahora es de tamaño 5
output = Linear3(x)  # output es de tamaño 1
```

Ejemplos de Módulos

Módulo	Entrada	Salida	Comentario
Lineal	Definida por usuario	Definida por usuario	Usualmente usada en la parte final de una red.
Sigmoide	No le importa	Misma que entrada	Función Activación tradicional
MaxPooling2D	(C_In, H, W)	(C_In, O(H), O(W))	Capa típicamente usada para manejar audio crudo.
Convolucional2D	(C_Out, H, W)	(C_out, O(H), O(W))	Capa típicamente usada para manejar imágenes o espectrogramas de audio.
ReLU	No le importa	Misma que entrada	Función Activación más usada en la práctica.

3. Entrenar Modelo

- Entrenar el modelo consiste en:
  1. Inicializar modelo con datos aleatorios o valores preentrenados.
  2. Entregar minibatch de datos al modelo para que éste genere un **output**.
  3. Comparar ese output con el output esperado (ground truth) usando la función de pérdida y obtener el **valor de la pérdida**.
  4. Hacer **backpropagation** para calcular los gradientes asociados a cada uno de los parámetros del modelo.
  5. **Actualizar los parámetros** del modelo usando el gradiente y la tasa de aprendizaje definida (SGD).
  6. Volver a 2 si es que no hemos entrenado por el número de épocas definido.

3. Entrenar Modelo

- En pseudo código sería:

```
for epoch from 1 to num_epochs:
   for x, y in train_dataset:
      # x, y son minibatch de datos entrada y salida esperada
      model_output = model(x)
      # funcion perdida es MSE o CrossEntropy
      loss = funcion_perdida(model_output,
   y)
      grads = backpropagation(loss)
      model = update_model(model, lr, grads)
```

4. Evaluar Modelo

- Evaluar el modelo consiste en determinar **qué tan bien está rindiendo nuestro modelo** usando algunas métricas.
- Métricas usuales:
  - **Pérdida**: queremos que sea lo más baja posible.
  - La **exactitud** en la predicción: queremos que llegue lo más cercano a 100%.
    - Sólo para problemas clasificación.

ESCUELA DE INGENIERÍA
FACULTAD DE INGENIERÍA

EDUCACIÓN PROFESIONAL

www.educacionprofesional.ing.uc.cl